M3 CLI Tool
Developer's Guide

M3CLI-03

February 2026

Version 1.0

Table of Contents

Preface

About This Guide

`Maestro CLI`, also known as `M3-CLI`, is a robust command-line interface (CLI) tool designed to provide a powerful alternative to user interfaces (UI). It allows you to perform cloud infrastructure management operations, as well as manage related assets, by sending respective calls to Maestro cloud management platform. The guide is aimed at Maestro users who extend or customize Maestro finctionality, ore perform integrations.

Target Audience

This guide is intended for:

- **Platform Engineers** who need to extend or customize M3-CLI functionality
- **DevOps Teams** who want to integrate M3-CLI into their automation workflows
- **System Administrators** who need to understand the internal workings of M3-CLI
- **Contributors** who wish to add new commands or modify existing ones
- **Integration Developers** who are building tools that interact with M3-CLI

When You Need This Guide

You should refer to this guide if you:

1. **Need to add new commands** to M3-CLI for your organization
2. **Want to customize command behavior** through integration plugins
3. **Are developing automation scripts** that leverage M3-CLI's architecture
4. **Need to understand** how M3-CLI processes commands and communicates with the Maestro3 platform
5. **Want to contribute** to the M3-CLI project
6. **Are troubleshooting** command behavior or creating custom output formatters

Related Documents

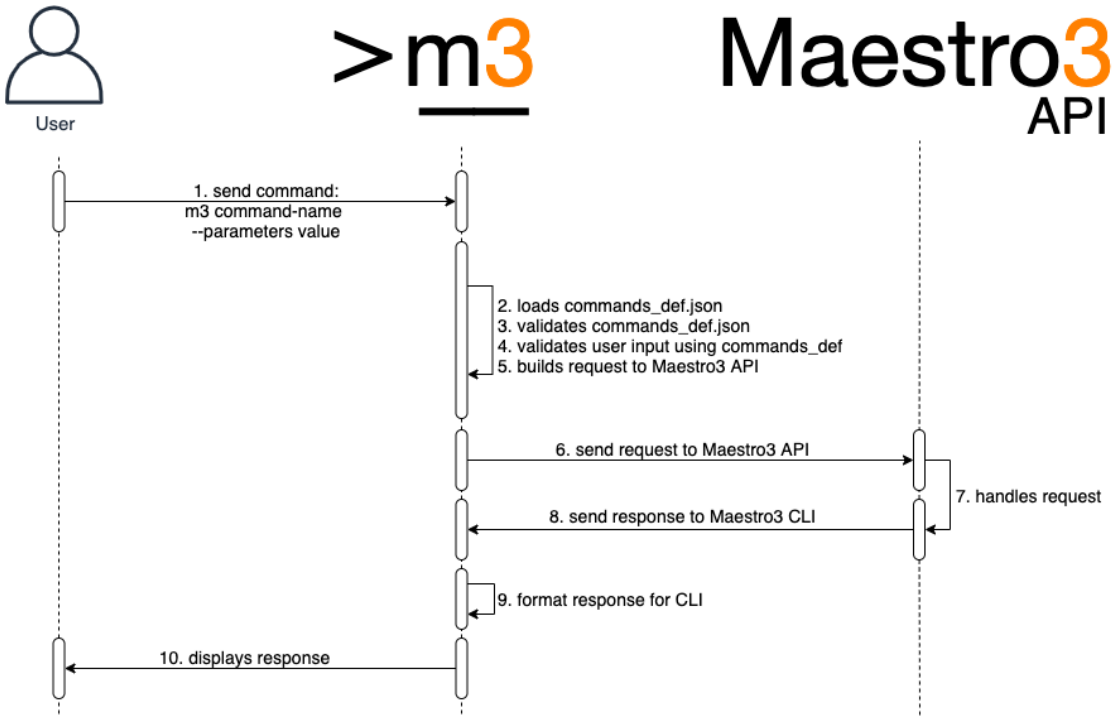
- For the information on Maestro CLI tool setup, configuration, and usage, refer to M3 User Guide.
- For the detailed information on the CLI commands, their parameters and usage, refer to M3-CLI Reference Guide.
- For the information on Maestro platform administering, to M3 Admin User Guide.

1. Architecture Overview

The `M3-CLI` tool is designed to provide a dynamic command-line interface based on command configurations declared in the `commands_def.json` file.

The tool DOES NOT perform any business logic related to management of resources that Maestro3 provides.

The sequence diagram is displayed below:



2. Commands Definition File

The commands definition file (`commands_def.json`) defines a set of commands, their groupings, and parameters that can be executed via CLI. Here is a full example of attributes found in the definition file:

```
{
    "groups": [
        "A name of a group of related commands"
    ],
    "domain_parameters": {
        "domain-parameter-name-1": {
            "alias": "param_alias",
            "api_param_name": "The name of the parameter that the server accepts to
form the response",
            "help": "Parameter description",
            "required": true,
            "validation": {
                "type": "string"
            }
        }
    },
    "commands": {
        "command-name-1": {
            "api-action": "REQUIRED. Mapping for M3API command",
            "help_file": true,                      // A Flag. If commands help
                                                    // stored into the file
            "help": "Explanation of what command does",
            "alias": "Alias for the command name. Could be used instead of command-
name-1",
            "integration_request": true,            // A Flag. Specify if the
                                                    // request will be processed

            "integration_response": true,           // A Flag. Specify if the
                                                    // response will be processed

            "integration_suffix": "",               // if specified, m3cli should
                                                    // build the plugin name
                                                    // according to the
                                                    // ${command_name}_
                                                    // ${integration_suffix}.py
                                                    // pattern
            "groups": [
                "group1",                           // A name of the group of
                                                    // commands to which this
                                                    // command relates

                "cli-<command-name>-help",          // Defines a unidirectional
                                                    // relation to the <command-
                                                    // name> command

                "email-<notification_type>-group",  // Defines a relation to the
                                                    // Maestro notifications of
                                                    // the type <notification_type>
            ],
            "parameters": {                         // Optional
                "inherited-param-from-domain": {
                    "parent": "Name of the domain parameter to inherit properties."
```

```
                },
                "param-name-1": {
                    "parent": "name",               // The name of the parent
                    "alias": "Alias for the parameter name. Could be used instead of
param-name-1",
                    "api_param_name": "The name of the parameter that the server
accepts to form the response",
                    "help": "REQUIRED. Explanation of what the parameter means",
                    "required": true,               // Set the parameter as a
                                                    // REQUIRED

                    "secure": true,                 // Hide the value of the
                                                    // parameter in logs. Allowed
                                                    // values: [true, false]

                    "validation": {                 // REQUIRED. Set of
                                                    // validation rules.

                        "type": "string/object",    // REQUIRED. The type of
                                                    // parameter. Allowed values:
                                                    // ['string', 'number',
                                                    // 'list', 'object', 'date',
                                                    // 'bool', 'file']

                        "allowed_values": [],       // A list of allowed values.
                                                    // Applicable to param of
                                                    // types 'string' and 'list'

                        "regex": "Regular Expression. Applicable to params of types:
'string', 'file'",
                        "regex_error": "A message to show if the regex check failed.
Applicable to params of types: 'string', 'file'",
                        "properties": {},           // jsonschema validation
                                                    // rules. Applicable to param
                                                    // of type: 'object';

                        "min_value": 0,             // number validation rule.
                                                    // Applicable only to type:
                                                    // 'number',

                        "max_value": 7,             // number validation rule.
                                                    // Applicable only to type:
                                                    // 'number',

                        "max_size_bytes": 1024,     // file validation rule.
                                                    // Applicable only to type:
                                                    // 'file',

                        "file_extensions": ['.txt'] // file validation rule.
                                                    // Applicable only to type:
                                                    // 'file'

                    },
                    "case": "upper/lower"           // convert a value to upper
                                                    // case automatically
                }
            },
```

```
            "output_configuration": {              // REQUIRED. Contains
                                                   // configuration for the
                                                   // output.

                "response_table_headers": [        // Required. Contains list
                  "header-1", "header-2"           // of attributes to display
                ],                                 // in output.
                "none": true,                      // Replace the response from
                                                   // server to "The command has
                                                   // been executed successfully"

                "nullable": true,                  // Set the flag 'nullable' in
                                                   // order to prevent hiding
                                                   // zeros and False values of
                                                   // number and boolean
                                                   // attributes.

                "multiple_table": true,            // A flag. If the response
                                                   // consists of several tables

                // The structure of the "response_table_headers" if the
                // "multiple_table" was specified
                "response_table_headers": [
                  {
                    // The display name of a table
                    "display_name": "Instance price model:",
                    // The name of the table that the server sends in the response
                    "name": "instancePriceModel",
                    // Contains list of attributes to display in output.
                    "headers": []
                  }
                ],
                // Optional. Used to alter appearance of data in columns.
                "headers_customization": {
                  // The header name
                  "the_name_of_header": {
                    // Custom name of the header
                    "header_display_name": "Custom name of the header",
                    // A flag. Disables automatic conversion of string-values to
                    // numbers.
                    "disable_numparse": true,
                    // A flag. Disables automatic alignment of list values to a
                    // column.
                    "prevent_list_formatting": true
                  }
                },
              "unmap_key": "Contains 'key' to extract response if needed."
            }
        }
    },
    "version": "major.minor"     // see the versioning rules below
}
```

Tip: Command help values can be stored in a file named `commands_help.py`. The format for this storage type is Python native. Example:

```
run-instance = """
Use this command to run an instance.
```

```
Examples:
1. Describes all available instances for a certain tenant in the specified region
    m3 run-instance -tn <tenant-name> -r <region> -iname <instance-name> -shname
<shape-name> -key <key-name> -count <number-of-instances>
"""
```

3. Related Commands Section in “full-help” Contents

The related commands section is constructed automatically based on the `groups` attribute of each command. Related commands are displayed as:

```
Related commands:

1. Deletes a schedule from your tenant's schedule library:
        m3 delete-schedule -r <region> -tn <tenant> -n <name>
```

There are two ways a command can be added to the list of related commands of another command.

Approach 1: Bidirectional Group Relations

Declare a group of commands in the `group` attribute in the root of the `commands_def` file, and then assign it to all commands within that group. All commands within the group share a bidirectional relation with each other.

Example:

```
{
  "groups": ["group_A"],
  "commands": {
    "first_command": {
      "groups": ["group_A"],
      // ...
    },
    "second_command": {
      "groups": ["group_A"],
      // ...
    }
  }
}
```

With this configuration, `first_command` will be in the list of related commands of `second_command` and vice versa.

Approach 2: Unidirectional Auxiliary Commands

Define a unidirectional relation between commands by adding the name of the other command to the list of groups with a special prefix and suffix (`cli-` and `-help`).

Example:

```
{
  "commands": {
    "first_command": {
      "groups": ["cli-second-command-help"],
      // ...
    },
    "second_command": {
      // ...
    }
  }
}
```

With this configuration, `first_command` will appear in the list of related commands of `second_command`. However, `second_command` will not be in the list of related commands of `first_command`.

4. Default Command Parameter Values

Default values for command parameters can be stored in the `m3.properties` file:

```
...
tenant-name = SFTL-SLCTL
region = SFTL-OPENSTACK-SLCTL
...
```

You can create several `m3.properties` files in different directories with different default values. To select the required parameters, change the current working directory to the directory containing the appropriate `m3.properties` file.

Alternatively, store default values for command parameters in the `default.cr` file:

```
{
    ...
    "tenant-name": "SFTL-SLCTL",
    "region": "SFTL-OPENSTACK-SLCTL",
    ...
}
```

Default parameter values from the `m3.properties` file take precedence over values from the `default.cr` file.

5. Integration Request/Response

Since version `2.1.0`, it is possible to process CLI input and server output manually using custom Python code. To use this feature, you need to create a plugin with the appropriate properties:

0. Create a directory named `plugins` at the path specified as the value for the environment variable `M3CLI_CONFIGURATION_FOLDER_PATH`.

1. Create a simple Python module (`filename.py`) and place it in the `plugins` directory mentioned above.

2. To modify CLI input, create a method inside your script named `create_custom_request` that receives one parameter, `request`:

```
def create_custom_request(request):
    return request
```

3. To modify CLI output, create a method inside your script named `create_custom_response` that receives `request` and `response` parameters:

```
def create_custom_response(request, response):
    return response
```

> **Tip:** The type of the return value in the functions described above should be the same as the received parameter (`request/response`)

4. After successful plugin creation, add the appropriate attributes to the command description in the file `commands_def.json`: `"integration_request": true` or `"integration_response": true`

6. Nullable Boolean Fields

Since version `3.41.7`, it is possible to display boolean fields even when their value is `False`. To use this feature:

Add the `nullable` field with the value `true` to the `output_configuration` section of the `commands_def.json` file:

```
  "output_configuration": {
          "nullable": true,
          "response_table_headers": [
            "region",
            "nativeName",
            "cloud",
            "active",
            "hidden"
          ]
        }
```

In the example above, if the value of the `active` field is `false`, the value in the table column will not be hidden.

7. Command Output Formatting

When the CLI receives a response with items that have empty attributes, these attributes will be dropped from the response (in table view only).

To add custom formatting to a response attribute, add the `headers_customization` parameter to `output_configuration`:

```
"output_configuration": {
    "response_table_headers": ["header-1", "header-2"],
    "headers_customization": {
      "header-1": {
        "disable_numparse": true,
        "prevent_list_formatting": true
      }
    }
}
```

- The `disable_numparse` setting disables automatic conversion of string values to numbers.
- The `prevent_list_formatting` setting disables automatic alignment of list values to a column.

8. Interactive Options

`Interactive options` is a feature that enables a command to fetch additional parameters from the server and either prompt a user to interactively provide values for these parameters or create a file with additional parameters (a `varfile`) for future use.

Fetching Additional Parameters

To enable a command to fetch additional parameters, add the following attribute to the command's definition:

```
"interactive_options": {
    // The name of parameters group
    "option_name": "params",
    // The name of server handler that returns the list of additional parameters
    "parameters_handler": "GET_SERVICE_VARIABLES_INFO",
    // The name of server handler that validates the list of additional parameters
    "validation_handler": "VALIDATE_SERVICE_VARIABLES",
}
```

If this feature is enabled, the command will: 1. Get additional parameters via the `parameters_handler` 2. Ask the user to fill in values for these parameters or provide a varfile 3. Validate the additional parameters 4. Send them back to the server as part of the original request

Generating Variable Files

To enable a command to create a varfile, add the following attribute to the command's definition:

```
"interactive_options": {
    // The name of server handler that returns the list of additional parameters
    "parameters_handler": "GET_SERVICE_VARIABLES_INFO",
    // Marks the command as a generator of a file with additional parameters
    // fetched with the "parameters_handler"
    "generate_varfile": true,
}
```

The `varfile` is a JSON file that contains all the additional parameters that the `parameter_handler` provides. The parameters are either filled with default values or have empty values.

M3-CLI Autocomplete

By default, `M3-CLI` includes autocomplete functionality that supports only Unix-based platforms (bash & zsh interpreters).

Activating Autocomplete

1. Create a virtual environment.

2. Install `M3-CLI`.

3. Create a symlink to the virtual environment:

```
sudo ln -s your_path_to_venv/bin/m3 /usr/local/bin/m3
```

4. Start a new terminal session.

5. Execute the command:

```
sudo m3 enable-autocomplete
```

6. Restart the terminal session.

Deactivating Autocomplete

1. Execute the command:

```
sudo m3 disable-autocomplete
```

2. Restart the terminal session.

Contents

Version History

Version	Date	Summary
v.1.0	February 9, 2029	Initial version finalized

View Changelog: https://github.com/Maestro-Cloud-Control/m3-cli/blob/main/CHANGELOG.md

The Documentation generated for: **Maestro CLI version: 4.158.3**
Commands version: 5.1